

Rapport du 14 au 30 Juillet

1. Résumé du rapport :

Au cours de ces deux dernières semaines, j'ai principalement travaillé sur une méthode pour automatiser l'exécution d'apprentissage de l'algorithme avec des paramètres différents.

En parallèle, j'ai également tenté d'analyser les causes de l'irrégularité de *MacSum Imprécis* en examinant les sources d'aléa dans le programme.

Enfin, j'ai implémenté les deux algorithmes décrits dans l'article ***Online Learning of Capacity-Based Preference Models***, afin de pouvoir les comparer ultérieurement à *MacSum Imprécis* et *MacSum*.

2. Irrégularité de MacSum Imprécis

Pour chercher à savoir pourquoi *MacSum Imprécis* était irrégulier, j'ai commencé par voir du côté des éléments "aléatoires" de l'algorithme, c'est à dire les paires de phrases et le paramètre Φ .

J'ai donc commencé par faire des tests avec un `var_phi` constant et des paires de phrases aléatoires.

J'ai ensuite fait des tests avec un `var_phi` aléatoire et des paires constantes.

Voici un tableau récapitulatif des résultats obtenues en faisant la moyenne de 5 exécution de l'algorithme à l'ordre FORT et FAIBLE, pour un φ aléatoires et des paires aléatoires, un φ constant et des paires aléatoires, un φ aléatoire et des paires constantes, et un φ constant et des paires constantes. Les simulations ont été réalisées avec le mode L2, 20 simulations différentes, et la base de données ***Housing***.

Type	Ordre	Préférence Recall	Préférence Precision	Indifférence Recall	Indifférence Precision	Total Recall
Hasard	Faible	0.399 ± 0.04	0.454 ± 0.039	0.488 ± 0.081	0.587 ± 0.115	0.421 ± 0.035
Hasard	Fort	0.76 ± 0.159	1.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.537 ± 0.118
Paires Fixes	Faible	0.279 ± 0.196	0.359 ± 0.261	0.342 ± 0.133	0.859 ± 0.199	0.327 ± 0.148
Paires Fixes	Fort	0.223 ± 0.251	0.455 ± 0.478	0.547 ± 0.492	0.626 ± 0.464	0.483 ± 0.341
φ Fixe	Faible	0.478 ± 0.137	0.514 ± 0.138	0.345 ± 0.079	0.674 ± 0.135	0.445 ± 0.079
φ Fixe	Fort	0.728 ± 0.114	1.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.487 ± 0.062

Le tableau montre que, sauf pour le cas des *Paires Fixes*, les modèles d'ordre FORT ont tendance à prédire que les paires sont uniquement **indifférentes**.

En revanche, pour l'ordre FAIBLE, on constate que le type ayant le recall le plus élevé est l'*Hasard*.

Malheureusement, je ne sais pas encore comment expliquer ces résultats. Si les résultats étaient

meilleurs pour les paires fixes dans tous les cas de figure (que ce soit pour l'ordre FORT ou FAIBLE), on pourrait penser que le modèle a un problème avec cette méthode de tirage des données. Ce n'est cependant pas le cas.

Dans un second temps, j'ai modifié dans l'algorithme les calculs utilisant Φ . Ainsi, au lieu de faire simplement la différence entre Φ et Φ' , on fait la moyenne des deux.

Type	Ordre	Préférence Recall	Préférence Precision	Indifférence Recall	Indifférence Precision	Total Recall
Moyenné	Faible	0.450 ± 0.102	0.5 ± 0.109	0.466 ± 0.114	0.62 ± 0.081	0.444 ± 0.061
Moyenné	Fort	0.742 ± 0.083	0.990 ± 0.023	0.011 ± 0.025	0.015 ± 0.034	0.516 ± 0.073
Classique	Faible	0.399 ± 0.04	0.454 ± 0.039	0.488 ± 0.081	0.587 ± 0.115	0.421 ± 0.035
Classique	Fort	0.76 ± 0.159	1.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.537 ± 0.118

A la lecture de ce tableau, on constate que faire la moyenne des Φ améliore les résultats de l'algorithme, que ce soit pour l'ordre FORT ou FAIBLE.

Cependant, on constate toujours que pour l'ordre FORT, l'algorithme ne prédit que les paires sont uniquement **indifférentes**.

Cependant, j'ai été pris de doutes une fois le tableau précédent dressé...

Vous m'avez demander de faire la "*moyenne des dérivée de q_phi* ", j'ai donc fait la moyenne de la soustraction de $\Phi(x)$ et de $\Phi(x')$.

Est-ce le bon calcul à faire ?

3. Implémentation des Algorithmes

Une fois les irrégularités étudiées, j'ai commencé à convertir en .c les algorithmes "1" et "2" présentés dans l'article *Online Learning of Capacity-Based Preference Models*.

Dans un premier temps, je vais faire un rappel rapide des deux algorithmes, puis je vais développer le calcul du gradient de l'erreur, avant finalement présenter les résultats obtenues.

3.1. L'algorithme RDA

Ce premier algorithme est un algorithme de type RDA (Red Dear Algorithm).

L'algorithme a la structure suivante :

Algorithm 1: RDA

```
1: procedure RDA( $\gamma, \lambda, T$ )
2:    $t \leftarrow 1$ 
3:    $m_1 \leftarrow (0, \dots, 0)$ 
4:
5:   while  $t < T$  do
6:      $\triangleright$  Receive pairwise example  $(x_t, y_t)$ 
7:      $\triangleright$  compute loss gradient  $g_t \in \partial \ell_t(m_t)$ 
8:      $\triangleright$  update average gradient  $\bar{g}_t$ 
9:      $m_{t+1} \leftarrow -\frac{\sqrt{t}}{\gamma} [|\bar{g}_t| - \lambda]_+ \text{sign}(\bar{g}_t)$ 
10:  end
11:  return  $m_T$ 
12: end
```

Avec les paramètres suivants :

- λ est un hyperparameter de taille unique.
- γ est un hyperparameter de taille unique.
- T est le nombre d'iterations.

3.2. L'algorithme ADMM-RDA

Algorithm 2: RDA with Bregman Projection

```

1: procedure ADMM-RDA( $\gamma, \lambda, \rho, T$ )
2:    $t \leftarrow 1$ 
3:    $m_1, \mu_1, z_1 \leftarrow (0, \dots, 0)$ 
4:
5:   while  $t < T$  do
6:      $\triangleright$  receive pairwise example  $(x_t, y_t)$ 
7:      $\triangleright$  compute loss gradient  $g_t \in \partial \ell_t(m_t)$ 
8:      $\triangleright$  update average gradient  $\{g\}_t$ 
9:      $g_t^\mu \leftarrow g_t - B^T(\bar{\mu}_t - \rho(B\bar{m}_t - \bar{z}_t))$ 
10:     $m_{t+1} \leftarrow -\frac{\sqrt{t}}{\gamma} [|\bar{g}_t| - \lambda]_+ \text{sign}(\bar{g}_t)$ 
11:     $z_{t+1} \leftarrow -\left[\frac{\mu_t}{\rho} - Bm_{t+1}\right]_+$ 
12:     $\mu_{t+1} \leftarrow \mu_t - \rho(Bm_{t+1} - z_{t+1})$ 
13:  end
14:  return  $m_T$ 
15: end

```

Avec les paramètres suivants :

- λ est un hyperparameter qui contrôle le degrés de régulation.
- γ est un hyperparameter, mais je sais pas à quoi il sert...
- ρ est un hyperparameter qui contrôle le niveau de pénalisation.
- T est le nombre d'itération.

La matrice B est construite de la même façon que pour la génération d'une ligne de critère avec la manière *GAME* : Pour $n = 4$,

$$B = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

J'ai également modifié la fonction de création de la matrice B afin d'y intégrer la possibilité de régler le *k-additivité*, de la même manière que pour la fonction *GAME*.

3.3. Fonction d'Erreur

Le calcul du loss gradient g_t est présentée sous la forme $\partial \ell_t(m_t)$.

Elle comprend une fonction l_t , cette dernière était :

$$l_t = \begin{cases} [\delta - \langle m, \Phi(x_t) - \Phi(y_t) \rangle]_+ & \text{si } t \in P \\ [|\langle m, \Phi(x_t) - \Phi(y_t) \rangle| - \delta]_+ & \text{si } t \in I \end{cases}$$

On doit donc calculer le gradient de cette fonction. Voici la retranscription de la fonction en un algorithme (la fonction C se trouve dans le fichier `algorithm.c`) :

Algorithm 3: Fonction Loss Gradient

```
1: procedure COMPUTE GRADIENT( $x_t, y_t, m, \delta, P$ )
2:    $\triangleright$  Initialize the used variables
3:    $s \leftarrow 0$ 
4:    $\text{coeff} \leftarrow 0$ 
5:
6:    $s \leftarrow \langle m, (x_t - y_t) \rangle$ 
7:
8:   if  $P = \text{true}$  and  $s < \delta$  then
9:      $\text{coeff} \leftarrow -1$ 
10:
11:   else if  $P = \text{false}$  and  $|s| > \delta$  then
12:      $\text{coeff} \leftarrow \text{sign}(s)$ 
13:
14:   else
15:      $\text{coeff} \leftarrow 0$ 
16:
17:   end
18:
19:    $\triangleright g_{\text{out}}$  will be different of 0 only if there is a violation
20:    $g_{\text{out}} \leftarrow \text{coeff} * (x_t - y_t)$ 
21:
22:   return  $g_{\text{out}}$ 
23: end
```
